8. Develop a java program to prepare a simulated data set with unique instances.

```
import java.io.*;
import java.util.*;

public class Main {
public static void main(String[] args) throws Exception {

BufferedReader br = new BufferedReader(
new FileReader("C:\\Users\\Lenovo\\OneDrive\\Desktop\\iris.csv")
);

String line;
Set<String> uniqueData = new LinkedHashSet<>();

String header = br.readLine();

while ((line = br.readLine()) != null) {
uniqueData.add(line);
}

br.close();

System.out.println("\n      SIMULATED DATASET WITH UNIQUE INSTANCES");
System.out.println("==========================================
==============");
```

```
System.out.printf("%-14s %-14s %-14s %-14s %-15s%n",
"Sepal Length", "Sepal Width", "Petal Length",
"Petal Width", "Species");
System.out.println("-------------------------------------------------------------");

int count = 0;

for (String data : uniqueData) {
String[] row = data.split(",");

if (row.length >= 5) {
System.out.printf("%-14s %-14s %-14s %-14s %-15s%n",
row[0], row[1], row[2], row[3], row[4]);
count++;
}

if (count == 10)
break;
}

System.out.println("================================================
==============");
System.out.println("Total Unique Instances: " + uniqueData.size());
}
}Output:
C:\Users\Lenovo\OneDrive\Desktop\hell>java Main
```

SIMULATED DATASET WITH UNIQUE INSTANCES

==========

Sepal Length Sepal Width Petal Length Petal Width Species

5.1 3.5 1.4 0.2 Iris-setosa

4.9 3 1.4 0.2 Iris-setosa

4.7 3.2 1.3 0.2 Iris-setosa

4.6 3.1 1.5 0.2 Iris-setosa

5 3.6 1.4 0.2 Iris-setosa

5.4 3.9 1.7 0.4 Iris-setosa

4.6 3.4 1.4 0.3 Iris-setosa

5 3.4 1.5 0.2 Iris-setosa

4.4 2.9 1.4 0.2 Iris-setosa

4.9 3.1 1.5 0.1 Iris-setosa

==========

Total Unique Instances: 147

9. Develop a Python program to generate frequent item sets / association rules using Apriori algorithm

Commands:

pip install pandas mlxtend

program

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

df = pd.read_csv("iris.csv")

# Convert numerical values into categories
```

```
for col in df.columns[:4]:
df[col] = pd.cut(df[col], bins=3, labels=["Low", "Medium", "High"])

# Convert rows into transactions
data = pd.get_dummies(df.astype(str))

# Find frequent itemsets
freq = apriori(data, min_support=0.1, use_colnames=True)
print("Frequent Itemsets:")
print(freq)

# Generate association rules
rules = association_rules(freq, metric="confidence", min_threshold=0.6)
print("\nAssociation Rules:")
print(rules[["antecedents", "consequents", "support", "confidence", "lift"]])
```

output

Frequent Itemsets:

	support	itemsets
0	0.133333	frozenset({sepallength_High})
1	0.393333	frozenset({sepallength_Low})
2	0.473333	frozenset({sepallength_Medium})
3	0.100000	frozenset({sepalwidth_High})
4	0.313333	frozenset({sepalwidth_Low})
..	...	...
115	0.140000	frozenset({petallength_Medium, sepalwidth_Medi...

116 0.240000 frozenset({petallength_Low, sepallength_Low, p...

117 0.100000 frozenset({sepallength_Medium, petallength_Med...

118 0.100000 frozenset({sepallength_Medium, petalwidth_High...

119 0.120000 frozenset({sepallength_Medium, sepalwidth_Medi...

[120 rows x 2 columns]

Association Rules:

antecedents ...   lift

0   frozenset({sepallength_High}) ... 2.771739

1   frozenset({sepallength_High}) ... 2.608696

2   frozenset({sepallength_High}) ... 2.550000

3   frozenset({sepallength_Low}) ... 1.068952

4   frozenset({petallength_Low}) ... 2.389831

..   ..

371 frozenset({sepalwidth_Medium, petallength_Medi... ... 3.229665

372 frozenset({sepalwidth_Medium, class_Iris-versi... ... 3.146853

373 frozenset({sepalwidth_Medium, petallength_Medi... ... 3.214286

374 frozenset({sepalwidth_Medium, petalwidth_Medium}) ... 3.452685

375 frozenset({sepalwidth_Medium, class_Iris-versi... ... 3.557312

[376 rows x 5 columns]

10. Develop a program to calculate chi-square value using Python/R. Report your observation.

Command

```
pip install pandas scipy
```

program

```
import pandas as pd
from scipy.stats import chi2_contingency

df = pd.read_csv("iris.csv")

# Create a contingency table
table = pd.crosstab(df.iloc[:, 4], pd.cut(df.iloc[:, 2], bins=3))
```

```
# Calculate chi-square value
chi2, p, dof, expected = chi2_contingency(table)

print("Contingency Table:")
print(table)
print("\nChi-square value:", round(chi2, 2))
print("P-value:", round(p, 4))
print("Degrees of freedom:", dof)

if p < 0.05:
print("Observation: Significant association exists.")
else:
print("Observation: No significant association exists.")
```

output

```
Contingency Table:
petallength      (0.994, 2.967]  (2.967, 4.933]  (4.933, 6.9]
class
Iris-setosa                50               0             0
Iris-versicolor             0              48             2
Iris-virginica              0               6            44

Chi-square value: 256.52
P-value: 0.0
Degrees of freedom: 4
Observation: Significant association exists.
```

11. Develop a program of Naive Bayesian classification using Python/R programming language.

Commands:

```
pip install pandas scikit-learn
```

program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

df = pd.read_csv("iris.csv")
```

```
X = df.iloc[:, :4]
y = df.iloc[:, 4]

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("\n========== NAIVE BAYES CLASSIFICATION ==========")
print(f"{'Actual Species':<22}{'Predicted Species':<22}")
print("-" * 44)

for actual, predicted in zip(y_test, pred):
print(f"{actual:<22}{predicted:<22}")

print("-" * 44)
print(f"Accuracy: {accuracy_score(y_test, pred) * 100:.2f}%")
print("================================================")
```

output:

```
========== NAIVE BAYES CLASSIFICATION ==========
Actual Species        Predicted Species
----------------------------------------
Iris-versicolor       Iris-versicolor
```

Iris-setosa	Iris-setosa
Iris-virginica	Iris-virginica
Iris-versicolor	Iris-versicolor
Iris-versicolor	Iris-versicolor
Iris-setosa	Iris-setosa
Iris-versicolor	Iris-versicolor
Iris-virginica	Iris-virginica
Iris-versicolor	Iris-versicolor
Iris-versicolor	Iris-versicolor
Iris-virginica	Iris-virginica
Iris-setosa	Iris-setosa
Iris-setosa	Iris-setosa
Iris-setosa	Iris-setosa
Iris-setosa	Iris-setosa
Iris-versicolor	Iris-versicolor
Iris-virginica	Iris-virginica
Iris-versicolor	Iris-versicolor
Iris-versicolor	Iris-versicolor
Iris-virginica	Iris-virginica
Iris-setosa	Iris-setosa
Iris-virginica	Iris-virginica
Iris-setosa	Iris-setosa
Iris-virginica	Iris-virginica
Iris-virginica	Iris-virginica
Iris-virginica	Iris-virginica
Iris-virginica	Iris-virginica

Iris-virginica Iris-virginica

Iris-setosa Iris-setosa

Iris-setosa Iris-setosa

Accuracy: 100.00%

==========

12. Implement a Java/R program to perform Apriori algorithm

```
import java.io.*;
import java.util.*;

public class Main {
public static void main(String[] args) throws Exception {
BufferedReader br = new BufferedReader(
new FileReader("C:\\Users\\Lenovo\\OneDrive\\Desktop\\iris.csv"));

List<Set<String>> data = new ArrayList<>();
String line;
br.readLine();

while ((line = br.readLine()) != null) {
String[] r = line.split(",");
Set<String> row = new TreeSet<>();

for (int i = 0; i < 4; i++) {
double v = Double.parseDouble(r[i].trim());
String cat = v < 3 ? "Low" : v < 5 ? "Medium" : "High";
```

```
row.add("F" + (i + 1) + "=" + cat);
}

row.add("Species=" + r[4].trim());
data.add(row);
}
br.close();

Set<String> items = new TreeSet<>();
for (Set<String> row : data)
items.addAll(row);

List<Set<String>> level = new ArrayList<>();
for (String item : items)
level.add(new TreeSet<>(Arrays.asList(item)));

int shown = 0, k = 1;
System.out.println("\n===== APRIORI: FIRST 10 FREQUENT ITEMSETS =====");

while (!level.isEmpty() && shown < 10) {
List<Set<String>> next = new ArrayList<>();

for (Set<String> s : level) {
int count = 0;
for (Set<String> row : data)
if (row.containsAll(s)) count++;
```

```
double support = (double) count / data.size();

if (support >= 0.1) {
System.out.printf("%-45s Support: %.2f%n", s, support);
shown++;

if (shown == 10) break;

for (String item : items) {
Set<String> candidate = new TreeSet<>(s);
candidate.add(item);
if (candidate.size() == k + 1 &&
!next.contains(candidate))
next.add(candidate);
}
}
}

if (shown == 10) break;

level = new ArrayList<>();
for (Set<String> s : next) {
int count = 0;
for (Set<String> row : data)
if (row.containsAll(s)) count++;
```

```
if ((double) count / data.size() >= 0.1)
level.add(s);
}
k++;
}

System.out.println("-----------------------------------------");
System.out.println("Total records processed: " + data.size());
System.out.println("Itemsets displayed: " + shown);
}
}
```

Outpt

```
===== APRIORI: FIRST 10 FREQUENT ITEMSETS =====
[F1=High]                       Support: 0.85
[F1=Medium]                         Support: 0.15
[F2=Low]                        Support: 0.38
[F2=Medium]                         Support: 0.62
[F3=High]                       Support: 0.31
[F3=Low]                        Support: 0.33
[F3=Medium]                         Support: 0.36
[F4=Low]                        Support: 1.00
[Species=Iris-setosa]                   Support: 0.33
[Species=Iris-versicolor]                   Support: 0.33
-----------------------------------------
```

Total records processed: 150

Itemsets displayed: 10

13. Develop a R program to cluster your choice of data using simple k-means algorithm using JDK

```
import java.io.*;
import java.util.*;

public class Main {
public static void main(String[] args) throws Exception {
BufferedReader br = new BufferedReader(
new FileReader("C:\\Users\\Lenovo\\OneDrive\\Desktop\\iris.csv"));

List<double[]> data = new ArrayList<>();
String line;
br.readLine(); // Skip header

while ((line = br.readLine()) != null) {
String[] r = line.split(",");
double[] x = new double[4];
for (int i = 0; i < 4; i++)
x[i] = Double.parseDouble(r[i].trim());
data.add(x);
}
br.close();

int k = 3, n = data.size();
double[][] c = new double[k][4];
```

```
for (int i = 0; i < k; i++)
c[i] = data.get(i).clone();

int[] cluster = new int[n];

for (int iter = 0; iter < 10; iter++) {
for (int i = 0; i < n; i++) {
double min = Double.MAX_VALUE;
for (int j = 0; j < k; j++) {
double d = 0;
for (int f = 0; f < 4; f++)
d += Math.pow(data.get(i)[f] - c[j][f], 2);
if (d < min) {
min = d;
cluster[i] = j;
}
}
}

double[][] sum = new double[k][4];
int[] count = new int[k];

for (int i = 0; i < n; i++) {
int cl = cluster[i];
count[cl]++;
```

```
for (int f = 0; f < 4; f++)
sum[cl][f] += data.get(i)[f];
}

for (int j = 0; j < k; j++)
if (count[j] > 0)
for (int f = 0; f < 4; f++)
c[j][f] = sum[j][f] / count[j];
}

System.out.println("\n===== K-MEANS CLUSTERING =====");
System.out.printf("%-12s %-12s%n", "Cluster", "Instances");
System.out.println("------------------------");

int[] count = new int[k];
for (int cl : cluster)
count[cl]++;

for (int j = 0; j < k; j++)
System.out.printf("%-12d %-12d%n", j + 1, count[j]);

System.out.println("------------------------");
System.out.println("Total instances: " + n);
System.out.println("K = " + k);
}
}
```

Goui

===== K-MEANS CLUSTERING =====

Cluster Instances

---

1 40

2 60

3 50

Total instances: 150

K = 3

14. Develop a program of cluster analysis using simple k-means algorithm Python/R programming language.

Comm

pip install pandas scikit-learn

```
import pandas as pd

from sklearn.cluster import KMeans


# Load Iris dataset

df = pd.read_csv("iris.csv")


# Select four numerical features
```

```
X = df.iloc[:, :4]

# Apply K-Means
model = KMeans(n_clusters=3, random_state=42, n_init=10)
df["Cluster"] = model.fit_predict(X)

# Display first 10 instances
print("===== K-MEANS CLUSTERING =====")
print(df[["Cluster"]].head(10).to_string(index=False))

print("\nCluster Analysis:")
print(df["Cluster"].value_counts().sort_index())

print("\nTotal instances:", len(df))
```

out

```
===== K-MEANS CLUSTERING =====
Cluster
1
1
1
1
1
1
1
1
1
1
```

1

Cluster Analysis:

Cluster

0 62

1 50

2 38

Name: count, dtype: int64

Total instances: 150

15. Develop a program to compute/display dissimilarity matrix (for your own dataset containing at least four instances with two attributes) using Python

Com

pip install pandas scipy

pro

import pandas as pd

from scipy.spatial.distance import cdist

Load Iris dataset

df = pd.read_csv("iris.csv")

```
# Select two attributes and four instances
data = df.iloc[:4, :2].values

# Compute dissimilarity matrix using Euclidean distance
matrix = cdist(data, data, metric="euclidean")

print("Selected Data:")
print(df.iloc[:4, :2].to_string(index=False))

print("\nDissimilarity Matrix:")
print(pd.DataFrame(matrix).round(2).to_string(index=False, header=False))
```

out

Selected Data:

sepallength	sepalwidth
5.1	3.5
4.9	3.0
4.7	3.2
4.6	3.1

Dissimilarity Matrix:

0.00 0.54 0.50 0.64

0.54 0.00 0.28 0.32

0.50 0.28 0.00 0.14

0.64 0.32 0.14 0.00

16. Visualize the datasets using matplotlib in python/R.(Histogram, Box plot, Bar chart, Pie chart etc.,)

Co

```
pip install pandas matplotlib

pro

import pandas as pd

import matplotlib.pyplot as plt


# Load Iris dataset

df = pd.read_csv("iris.csv")
```

```
# Create four charts
plt.figure(figsize=(10, 8))

# 1. Histogram
plt.subplot(2, 2, 1)
plt.hist(df.iloc[:, 0], bins=10, edgecolor="black")
plt.title("Histogram - Sepal Length")
plt.xlabel("Sepal Length")
plt.ylabel("Frequency")

# 2. Box Plot
plt.subplot(2, 2, 2)
plt.boxplot(df.iloc[:, 0])
plt.title("Box Plot - Sepal Length")
plt.ylabel("Sepal Length")

# 3. Bar Chart
plt.subplot(2, 2, 3)
df.iloc[:, 4].value_counts().plot(kind="bar")
plt.title("Bar Chart - Species")
plt.xlabel("Species")
plt.ylabel("Count")
plt.xticks(rotation=30)

# 4. Pie Chart
plt.subplot(2, 2, 4)
```

```
df.iloc[:, 4].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Pie Chart - Species")
plt.ylabel("")

plt.tight_layout()
plt.show()
```

out

